

WSQ_系统设计说明书

依据标准：计算机软件文档编制规范

项目名称：陪伴型 AI 聊天机器人系统

开发团队：星途团队

版本号：V1.0

1 引言

1.1 编写目的

本说明书旨在全面阐述陪伴型 AI 聊天机器人系统的总体架构、分层设计、功能模块、接口规范、业务流程、运行环境及系统设计。本文档为项目开发、测试、验收、维护、归档的权威技术依据，保证系统设计完整、准确、层级清晰，满足软件开发规范化要求。

1.2 项目背景

当前市面通用 AI 聊天机器人存在人设固定、对话风格单一、无法自定义个性化陪伴场景等问题，难以满足用户私人情感陪伴、专属对话模拟的需求。为解决上述问题，本项目基于前后端分离架构，开发一款支持自定义人设、自定义对话风格、语料导入、多轮连贯对话的轻量化陪伴型 AI 聊天机器人系统，具备高自由度、高拟人度、易部署、易扩展的特点。

1.3 术语、定义和缩写

为保证全文统一规范，所有缩写统一定义如下：

- HTML**：超文本标记语言，用于构建网页前端结构。
- CSS**：层叠样式表，用于实现页面样式与布局渲染。
- JavaScript/JS**：前端脚本语言，实现页面交互逻辑。
- Node.js**：服务端 JavaScript 运行环境，用于搭建后端服务。
- Express**：基于 Node.js 的轻量级 Web 开发框架。
- API**：应用程序编程接口，用于系统模块间数据交互。

- 7. **CORS**: 跨域资源共享, 解决前后端分离项目跨域访问问题。
- 8. **JSON**: 轻量级数据交换格式, 为本系统统一数据传输格式。
- 9. **AI**: 人工智能, 本文特指大语言模型推理服务。
- 10. **Env**: 环境变量配置文件, 用于存储系统敏感配置信息。

1.4 参考资料

- [1] 《计算机软件文档编制规范 系统设计说明书》
- [2] Node.js、Express 官方技术开发文档
- [3] 火山方舟大模型 API 开发手册

2 总体设计

2.1 设计原则

本系统采用由宏观到微观、逐层深入的分层设计原则, 整体架构低耦合、高内聚, 功能模块划分清晰, 兼顾实用性、稳定性与可扩展性, 满足项目开发与后期迭代需求。

2.2 系统总体架构

系统采用标准三层分层架构, 层级逐级细化, 上层依赖下层, 结构规范统一:

- (1) **表现层 (前端)**: 负责用户界面展示、交互操作、参数采集、文件上传、页面状态管理, 不处理核心业务与敏感数据。
- (2) **业务逻辑层 (后端)**: 负责接口请求处理、参数校验、提示词组装、AI 模型调用、对话上下文管理、异常捕获处理, 是系统核心业务中枢。
- (3) **数据服务层**: 负责大模型 AI 推理服务调用、临时会话数据存储, 预留数据库持久化拓展接口。

2.3 系统运行环境

前端运行环境: 主流浏览器 (Chrome、Edge), 支持 HTML5 标准。

后端运行环境: Node.js 14.0 及以上版本, 支持 Windows、Linux 操作系统。

网络环境: 设备可正常访问公网大模型 API 服务。

2.4 系统整体功能概述

本系统实现个性化 AI 陪伴聊天核心业务, 支持机器人名称、性格人设、对话风格自定

义，支持手动输入语料、TXT 语料文件上传，具备消息发送、编辑、引用、多轮连贯对话、清空聊天记录、异常提示等完整功能，实现高度拟人化的智能陪伴效果。

3 功能模块详细设计

3.1 前端核心模块设计

(1) 界面布局模块：采用侧边栏配置区+主聊天区双栏布局，支持侧边栏折叠展开，适配 PC 端、移动端响应式布局，区分用户与 AI 消息气泡样式，界面整洁统一。

(2) 聊天交互模块：支持按钮发送、回车发送、Shift+Enter 换行；实现消息编辑、消息引用、页面自动滚动、防重复提交、一键清空聊天记录功能，交互体验流畅。

(3) 个性化配置模块：可视化配置机器人名称、性格特点、说话风格，配置实时生效，动态改变 AI 对话人设与语气。

(4) 语料管理模块：支持手动输入对话语料、上传 TXT 格式语料文件，自带文件格式校验、上传成功与失败提示，使 AI 精准模仿指定聊天风格。

(5) 状态管理模块：实现发送中加载状态、网络异常提示、接口错误提示，提升系统容错性与用户体验。

3.2 后端核心模块设计

(1) 接口服务模块：提供标准化 HTTP 接口，统一 JSON 返回格式，包含聊天对话接口、清空历史记录接口，为前端提供稳定数据服务。

(2) AI 智能处理模块：接收前端配置参数与用户消息，自动组装人设、风格、语料、历史对话为标准化提示词，调用大模型 API 并解析、封装有效回复数据。

(3) 上下文管理模块：基于用户唯一标识隔离对话数据，维护多轮对话上下文，保障对话连贯性，支持一键清空会话历史。

(4) 安全与异常模块：配置 CORS 跨域策略，敏感密钥存储于后端环境变量，杜绝前端信息泄露；全局捕获请求异常、接口异常，保证系统稳定运行。

4 系统接口设计

4.1 聊天对话接口

接口地址：/api/chat

请求方式：POST

接口功能：接收用户消息与机器人配置参数，调用大模型生成 AI 陪伴回复

请求参数：userId、userMessage、robotName、personality、chatStyle、chatCorpus

返回参数：code（状态码）、data（AI 回复内容）、msg（状态描述）

4.2 清空历史接口

接口地址：/api/clear-history

请求方式：POST

接口功能：清空指定用户的全部对话上下文与聊天记录

请求参数：userId

返回参数：code（状态码）、msg（状态描述）

5 系统业务流程

1. 用户进入系统前端页面，完成机器人名称、性格、风格、语料的个性化配置；
2. 用户输入对话内容并提交请求，前端校验数据后向后端接口发送请求；
3. 后端接收参数，校验合法性，整合人设、语料、历史对话生成标准化提示词；
4. 后端调用火山方舟大模型 API，获取 AI 推理回复；
5. 后端更新用户对话上下文，封装标准 JSON 数据返回前端；
6. 前端接收数据，渲染 AI 消息气泡，完成一轮对话交互；
7. 用户可随时清空聊天历史，后端重置对应用户的对话上下文。

6 系统性能与安全设计

6.1 性能设计

前端增加请求锁机制，防止重复提交；后端采用异步请求处理方式，提升接口响应效率；上下文轻量化存储，保障多轮对话流畅运行。

6.2 安全设计

所有 AI 密钥、模型 ID 等敏感信息存储于后端 .env 环境变量文件，前端无任何涉密数据；后端统一参数校验、全局异常捕获，有效规避非法请求与系统崩溃风险。

7 系统可扩展性设计

系统采用前后端分离解耦设计，可无缝接入 MongoDB 数据库实现数据持久化；可自

由切换各类大模型接口；可拓展用户登录、对话收藏、多机器人角色切换、历史记录查询等功能，整体架构无需重构，拓展性极强。